

```

GNU nano 0.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define INITIAL_SIZE 100
#define HISTORY_SIZE 10

int main() {
    char **history = malloc(HISTORY_SIZE * sizeof(char *));
    int history_count = 0;

    char *buffer = malloc(INITIAL_SIZE);
    int size = INITIAL_SIZE;

    if (buffer == NULL || history == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    while (1) {
        printf("\n$ ");

        if (fgets(buffer, size, stdin) == NULL)
            break;

        buffer[strcspn(buffer, "\n")] = '\0';

        /* Exit command */
        if (strcmp(buffer, "exit") == 0)
            break;

        /* Resize buffer if needed */
        if (strlen(buffer) >= size - 1) {
            size *= 2;
            char *temp = realloc(buffer, size);

            if (temp == NULL) {
                printf("Memory resize failed\n");
                free(buffer);
                break;
            }

            buffer = temp;
        }

        /* Store command in history */
        if (strlen(buffer) > 0) {
            if (history_count < HISTORY_SIZE) {
                history[history_count] = malloc(strlen(buffer) + 1);
                strcpy(history[history_count], buffer);
                history_count++;
            } else {
                free(history[0]);

                for (int i = 1; i < HISTORY_SIZE; i++)
                    history[i - 1] = history[i];

                history[HISTORY_SIZE - 1] =
                    malloc(strlen(buffer) + 1);
                strcpy(history[HISTORY_SIZE - 1], buffer);
            }
        }

        /* Display history */
        if (strcmp(buffer, "history") == 0) {
            printf("\nCommand History:\n");

            for (int i = 0; i < history_count; i++)
                printf("%d %s\n", i + 1, history[i]);
        }

        /* Release memory */
        for (int i = 0; i < history_count; i++)
            free(history[i]);

        free(history);
        free(buffer);

        printf("\nMemory released successfully.\n");

        return 0;
    }
}

```

Output

```
FileDescriptors Practical Practical_2 Skill exec fork program singal
[ubuntu@ubuntu:~/OSLab$ cd Skill
[ubuntu@ubuntu:~/OSLab/Skill$ nano skill13.c
[ubuntu@ubuntu:~/OSLab/Skill$ gcc skill13.c -o skill13
[ubuntu@ubuntu:~/OSLab/Skill$ ./skill13

[$ ls

[$ pwd

[$ gcc test.c

[$ history

Command History:
1  ls
2  pwd
3  gcc test.c
4  history
```